

# Práctica 11 — Clase 06

## Diseño de capa de acceso políglota para FlowOps

**Materia:** Ingeniería de Datos II

**Clase:** 06 — Acceso a bases NoSQL desde aplicaciones

**Vinculación con el TPO:** Esta práctica permite cerrar el diseño de acceso desde aplicación para el TPO FlowOps, definiendo cómo el backend se comunica con cada base NoSQL sin acoplar la lógica de negocio a una tecnología específica.

### 1. Contexto

Hasta este momento el TPO FlowOps fue incorporando distintos modelos de persistencia:

- Modelo documental para definiciones de procesos, formularios e instancias.
- Modelo columnar/tabular para eventos de auditoría y escritura masiva.
- Modelo de grafos para flujos, transiciones, dependencias y caminos.
- Modelo clave-valor para caché, sesiones, estado rápido y colas.
- Modelo de objetos para subdominios complejos, reglas, validaciones o comportamiento.

El problema ahora es diferente: **¿cómo accede una aplicación real a todos esos motores sin convertirse en un conjunto desordenado de conexiones, consultas y lógica duplicada?**

Esta práctica se concentra en diseñar una capa de acceso clara, defendible y mantenible.

### 2. Contenidos utilizados

Esta práctica utiliza contenidos vistos hasta la Clase 06:

- Arquitectura de aplicaciones con persistencia políglota.
- Drivers y conectores.
- Capa de acceso a datos.
- Repository Pattern.
- DAO.
- Servicios de negocio.
- API Gateway o API unificada.

- Separación de responsabilidades.
- Manejo de errores.
- Timeouts.
- Reintentos.
- Circuit breaker conceptual.
- Sincronización eventual.
- Consistencia entre motores.
- Decisiones tomadas en clases anteriores sobre MongoDB, Cassandra, Neo4j, Redis e IRIS.

No se exige:

- Implementar código funcional completo.
- Conectar realmente todos los motores.
- Crear una API productiva.
- Usar GraphQL.
- Implementar monitoreo avanzado.
- Hacer benchmarking real.

### 3. Objetivo de la práctica

El grupo deberá diseñar una **capa de acceso políglota para FlowOps**, especificando:

1. Qué servicio accede a cada base.
2. Qué operaciones expone cada repository o DAO.
3. Qué datos devuelve a la capa de negocio.
4. Qué errores debe manejar.
5. Qué operaciones deberían ser sincrónicas.
6. Qué operaciones podrían ser asincrónicas.
7. Qué dependencias deben quedar ocultas detrás de una interfaz.

### 4. Parte A — Arquitectura de aplicación

Dibujen o describan la arquitectura lógica de su aplicación FlowOps.

Estructura sugerida:

Frontend / Cliente de prueba

|

v

## API FlowOps

|

+--> Auth / Tenant Context

|

+--> Process Service

|     +--> ProcessDefinitionRepository

|

+--> Runtime Service

|     +--> InstanceRepository

|     +--> StateCacheRepository

|

+--> Task Service

|     +--> TaskRepository

|

+--> Audit Service

|     +--> EventLogRepository

|

+--> Graph Service

|     +--> ProcessGraphRepository

|

+--> Metrics / Connectivity Service

Completen:

| Capa                | Responsabilidad | Qué NO debería hacer |
|---------------------|-----------------|----------------------|
| API / Controller    |                 |                      |
| Service             |                 |                      |
| Repository / DAO    |                 |                      |
| Database Driver     |                 |                      |
| Cache Layer         |                 |                      |
| Event / Audit Layer |                 |                      |

Preguntas guía:

1. ¿Dónde se valida el `tenant_id`?
2. ¿Dónde se decide qué base usar?
3. ¿Dónde se transforma un documento en respuesta de API?
4. ¿Dónde se manejan timeouts y reintentos?
5. ¿Dónde se registran eventos de auditoría?

## 5. Parte B — Repositories o DAOs por motor

Definan al menos cinco repositories o DAOs para su TPO.

Ejemplo:

| Repository / DAO                         | Motor sugerido | Responsabilidad                         | Operaciones principales                                                           |
|------------------------------------------|----------------|-----------------------------------------|-----------------------------------------------------------------------------------|
| <code>ProcessDefinitionRepository</code> | MongoDB        | Guardar y leer definiciones de procesos | <code>save</code> ,<br><code>findPublished</code> ,<br><code>findByVersion</code> |
| <code>InstanceRepository</code>          | MongoDB / IRIS | Guardar estado persistente de instancia | <code>create</code> ,<br><code>updateStatus</code> ,<br><code>findById</code>     |

| Repository / DAO       | Motor sugerido | Responsabilidad                | Operaciones principales                                |
|------------------------|----------------|--------------------------------|--------------------------------------------------------|
| EventLogRepository     | Cassandra      | Registrar eventos inmutables   | appendEvent, findByInstance, findByTenantAndDate       |
| ProcessGraphRepository | Neo4j          | Consultar caminos del flujo    | findPaths, detectUnreachableSteps, validateTransitions |
| CacheRepository        | Redis          | Cache, sesiones, estado rápido | get, set, setWithTTL, delete                           |

Completen:

| Repository / DAO | Motor | Operaciones | Entrada | Salida | Riesgo principal |
|------------------|-------|-------------|---------|--------|------------------|
|                  |       |             |         |        |                  |

## 6. Parte C — Interfaces conceptuales

Diseñen interfaces conceptuales. No deben programarlas necesariamente, pero sí definir qué métodos debería exponer cada una.

Ejemplo:

```
interface ProcessDefinitionRepository {
```

```
    save(definition: ProcessDefinition): Promise<void>;
```

```
    findPublished(tenantId: string, processId: string): Promise<ProcessDefinition | null>;
```

```
    findByVersion(tenantId: string, processId: string, version: number): Promise<ProcessDefinition | null>;
```

```

}

interface EventLogRepository {

    append(event: FlowOpsEvent): Promise<void>;

    findByInstance(tenantId: string, instanceId: string): Promise<FlowOpsEvent[]>;

    findByTenantAndDate(tenantId: string, dateFrom: Date, dateTo: Date):
    Promise<FlowOpsEvent[]>;

}

```

Cada grupo debe definir al menos **tres interfaces**.

| Interfaz | Métodos | Motor detrás | Qué abstracción aporta |
|----------|---------|--------------|------------------------|
|          |         |              |                        |

## 7. Parte D — Flujo de ejecución de una operación

Describan el flujo completo para una operación importante del TPO.

Operación sugerida:

Iniciar una instancia de proceso.

Ejemplo conceptual:

1. La API recibe POST /tenants/{tenant\_id}/processes/{process\_id}/instances.
2. Auth valida usuario y tenant.
3. ProcessService solicita definición publicada al ProcessDefinitionRepository.
4. Si no está en caché, se consulta MongoDB.
5. RuntimeService crea una instancia inicial.
6. InstanceRepository persiste la instancia.

7. CacheRepository guarda estado actual con TTL.
8. EventLogRepository registra PROCESS\_STARTED.
9. ProcessGraphRepository valida el primer paso posible.
10. La API devuelve instance\_id, status y current\_node.

Completen para su propio TPO:

| Paso | Componente | Acción | Base involucrada | Error posible |
|------|------------|--------|------------------|---------------|
| 1    |            |        |                  |               |
| 2    |            |        |                  |               |
| 3    |            |        |                  |               |
| 4    |            |        |                  |               |
| 5    |            |        |                  |               |
| 6    |            |        |                  |               |
| 7    |            |        |                  |               |
| 8    |            |        |                  |               |

## 8. Parte E — Manejo de errores

Analicen los siguientes escenarios y definan una estrategia.

| Escenario                                           | Qué puede fallar | Estrategia | Qué se informa al usuario |
|-----------------------------------------------------|------------------|------------|---------------------------|
| MongoDB no responde al buscar definición de proceso |                  |            |                           |

| Escenario                                             | Qué puede fallar | Estrategia | Qué se informa al usuario |
|-------------------------------------------------------|------------------|------------|---------------------------|
| Redis no responde al consultar estado cacheado        |                  |            |                           |
| Cassandra no puede registrar evento                   |                  |            |                           |
| Neo4j no puede validar camino del flujo               |                  |            |                           |
| El usuario intenta completar dos veces la misma tarea |                  |            |                           |
| El tenant no existe o está deshabilitado              |                  |            |                           |

Preguntas guía:

1. ¿Qué fallas permiten continuar?
2. ¿Qué fallas obligan a cortar la operación?
3. ¿Qué fallas requieren reintento?
4. ¿Qué fallas requieren registrar auditoría?
5. ¿Qué datos se pueden reconstruir luego?
6. ¿Dónde usarían circuit breaker conceptual?

## 9. Parte F — Sincronización entre bases

Definan cómo se mantienen coherentes los datos entre bases.

Ejemplo:

Cuando se publica un proceso:

- MongoDB guarda la definición completa.
- Neo4j actualiza el grafo de pasos y transiciones.



- Redis invalida la definición cacheada anterior.

- Cassandra registra PROCESS\_PUBLISHED.

Completen:

| Evento de dominio  | Base principal | Bases derivadas | Sincrónico o asincrónico | Riesgo de inconsistencia |
|--------------------|----------------|-----------------|--------------------------|--------------------------|
| Proceso publicado  |                |                 |                          |                          |
| Instancia iniciada |                |                 |                          |                          |
| Tarea completada   |                |                 |                          |                          |
| Error de ejecución |                |                 |                          |                          |
| Proceso finalizado |                |                 |                          |                          |

## 10. Parte G — API unificada

Propongan al menos cuatro endpoints de su API FlowOps.

| Endpoint                                             | Método | Qué hace         | Servicios internos | Bases involucradas |
|------------------------------------------------------|--------|------------------|--------------------|--------------------|
| /api/v1/{tenant_id}/processes/{process_id}/instances | POST   | Inicia instancia |                    |                    |
| /api/v1/{tenant_id}/instances/{instance_id}          | GET    | Consulta estado  |                    |                    |

| Endpoint                                                        | Método | Qué hace           | Servicios internos | Bases involucradas |
|-----------------------------------------------------------------|--------|--------------------|--------------------|--------------------|
| <code>/api/v1/{tenant_id}/instances/{instance_id}/events</code> | GET    | Consulta auditoría |                    |                    |
| <code>/api/v1/{tenant_id}/tasks/{task_id}/complete</code>       | POST   | Completa tarea     |                    |                    |

Agreguen endpoints propios si su proceso lo requiere.

## 11. Parte H — Evidencia para el TPO

Definan qué evidencia podrían mostrar en la defensa final para demostrar que el acceso a datos está bien diseñado.

| Evidencia                       | Qué demuestra | Cómo se mostraría |
|---------------------------------|---------------|-------------------|
| Diagrama de arquitectura        |               |                   |
| Tabla repository-motor          |               |                   |
| Endpoint de inicio de instancia |               |                   |
| Consulta de eventos             |               |                   |
| Falla simulada de Redis         |               |                   |
| Log de auditoría                |               |                   |
| Captura de base NoSQL           |               |                   |

---

## 12. Entregable

Cada grupo entregará un documento de 3 a 5 páginas con:

1. Arquitectura lógica de aplicación.
2. Tabla de capas y responsabilidades.
3. Repositories o DAOs por motor.
4. Tres interfaces conceptuales.
5. Flujo completo de una operación.
6. Análisis de errores.
7. Estrategia de sincronización entre bases.
8. Endpoints de API.
9. Evidencia prevista para la defensa del TPO.

## 13. Puesta en común

Cada grupo tendrá 4 minutos para responder:

¿Qué decisión de acceso a datos reduce más la complejidad del TPO y por qué?

La respuesta debe mencionar:

- Un repository o DAO.
- Un motor NoSQL.
- Un error posible.
- Una decisión de abstracción.

## 14. Criterios de evaluación

| Criterio                                  | Peso |
|-------------------------------------------|------|
| Claridad de la arquitectura de aplicación | 20%  |
| Diseño de repositories/DAOs               | 25%  |
| Separación de responsabilidades           | 20%  |
| Manejo de errores y sincronización        | 20%  |
| Aplicación al TPO y defensa oral          | 15%  |